Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	**Semester:**	1st
Name of the Student:	D. Harsh Kumar	**Roll Number:**	A24126552136
Name of the Lab:	FS	**Timings:**	9.40 am to 12:10 pm

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
11/7/26	1	4	4	5	1	14	10	
8/7/26	2	4	4	5	1	14	10	
15/7/26	3	4	4	5	1	14	10	
22/7/26	4	3	4	5	1	13	10	
29/7/26	5	4	4	5	1	14	10	
5/8/26	6	4	4	5	1	14	10	
12/8	7	4	4	5	1	14	10	
19/8	8	4	4	5	1	14	10	
2/9	9	4	4	5	1	14	10	
9/9	10	4	4	5	1	14	10	

Name of the Faculty: Prasanna Priya

Faculty Signature:

Introduction to HTML and Css

* HTML (Hyper Text Markup Language) is the standard markup language used to Create and the Structure content on Webpages it defines that basic Structure of a Webpage using elements and tags such as headings, Paragraphs, Links, images lists, tables, forms and other multimedia Content, HTML documents are interepted by web browsers to display the Content in a Structured and an meaningful manner HTML Provides the Structure and Semantic organitation of a Webpage.

* Css (cascading Style Sheets) is a Stylesheet language used to Control the Presentation and visual apperance of HTML documents it is used to define Properties Such as Colours, fronts, background, Spacing, borders allignment, dimension and layouts Css also supports responsive design, allowing Webpages to adapt to different Screen Sizes and devices Css can be applied using inline internal or External style sheets.

* HTML and Css work together to develop Webpg, Whese html Provides the structure and Content, While Css Provides the Styling layout visual Presentation the Combination of both forms are foundation of modern web enables user friendly web interfaces.

Difference between MERN Stack and full Stack:

* MERN Stack is an javascript based technology stack

Used for developing modern web application. MEAN Stands for MongoDB, Express.js, React.js, Node.js. MongoDB is used for database management Express.js - handles the backend framework and API's React.js - develop frontend user interface and node.js provide the server side runtime environment.

* Full Stack Development refers to development of both the frontend backend of web application along with D.B integration

A full stack worker or an developer works with user interface and implementing the server side bgc managing databases.

HTML Components:

HTML tags — Define the structure and Element of a webpage.

Headings — Used to display titles & heading using <h1> to <h6>

Paragraphs — Used to display text using <p> tag

Links — Used to navigate b/w webpages of using <a> tag

images — used to display images using <img> tag

lists — used to create ordered & unordered lists using <ol>, <ul>,

Tables — Used to organize data in rows & columns

Forms — Used to collect users i/p using elements as text box, button, checkbox.

div & span — Used to group and diag organize webpage content.

Semantic Elements are <header>, <nav>, <select>, <article>, <footer>

Provide meaningful webpage structure.

CSS Components

Selections – Select HTML elements to supply styles

Properties – Define visual characteristics of elements

Values – Specify setting of CSS property

Colors – Define text, bg, border colours

Font – Control font family, size style & weight

Background – define the set bg colors, img

Borders – define border style, width radius

Margin – controls space outside element

Padding – space inside element

Layout – Control the arrangement of elements using flexbox, grid and positioning.

HTML vs HTML5

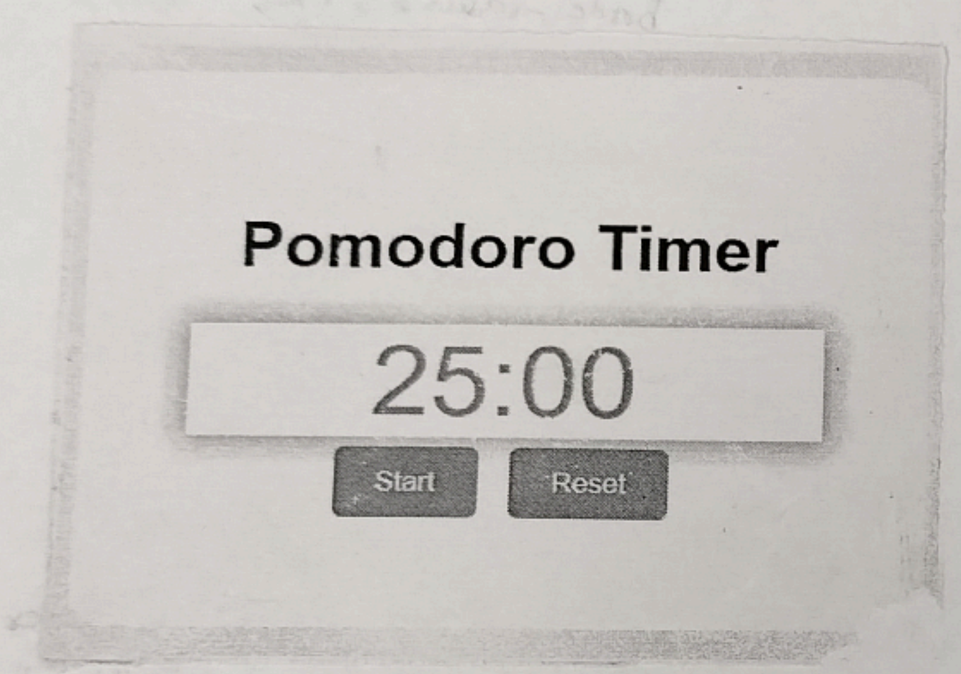
HTML	HTML5
Older version of html	latest major version of html
Limited multimedia support	support audio & video
Uses External plugins for multimedia	No external plugs need
Has fewer Semantic tags	Provides more Semantic tags.
NO <canvas> element	support <canvas>
Limited web storage	support local storage
Complex doctype declaration	simple <!doctype> HTML

CSS V/s CSS3

CSS	CSS3
Earlier version of CSS	Adv version of CSS
Basic styling Features	Adv styling features
Limited animation	support animation and transitions.
No flexible layout system	Supports flexbox and grid
Limited visual effects	Supports shadows, gradients, effects.

13-7 Design an static Webpage using html & css Pomodoro timer, using html css. Exp-2

```
<!doctype html>
<html>
<head>
    <title> Pomodoro timer </title>
    <style>
        body{
            font-family: Arial;
            background: #f5f5f5;
            text-align: center;
            Padding: 50px;
            }
    timer{
            width: 300px;
            margin: auto;
            Padding: 30 px;
            background: white;
            border radius: 15 px;
            box-shadow: 0 0 10px gray;
        }
    h1{
        Color: #074C3C;
      }
    timer{
        font-size: 50 px;
        margin: 20 px;
    }
    button{
        Padding: 10px 20px;
        margin: 5px;
        border: none;
        border-radius: 5 px;
        background: #c74c3c;
        Colour: white;
      }
    </style>
    </head>
    <body>
        <div class "timer">
            <h1> Pomodoro Timer </h1>
            <div class = "time" > 25:00</div>
            <button> start </button>
            <button> reset </button>
        </div>
    </body>
    </html>
```

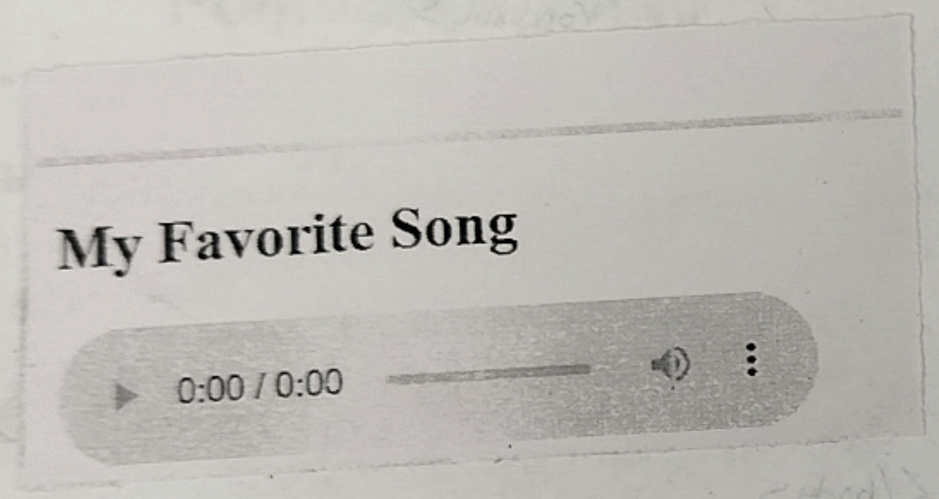


1) Design a HTML code using audio tags

```
<!Doctype html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My favorite song </h2>
    <audio control>
        <source src="song.mp3" type="audio/mpeg">
            Your browser does not suppor the
            audio tag

    </audio>
</body>
</html>
```

Steps:

* Create a new html file
* Add basic html structure
* Add heading using <h2> tag
* Insert <audio> tag with control attribute
* Specify the audio file using <source> tag
* Save, open Html file in web browsers
* Play audio using display controls



ii) Design a html code Using video tags.

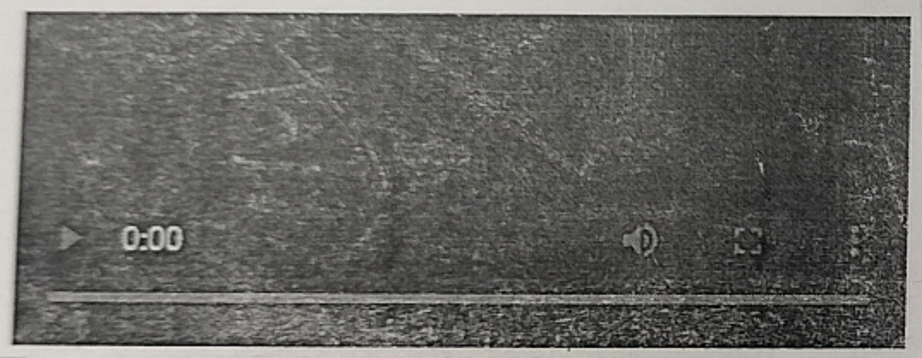
Code:

```
<!Doctype html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2>My video </h2>
    <video width = "400" controls>
    <source src = "video.mp4 "type = "video/mp4">
    </video>
</body>
</html>
```

Steps:

- Create a new html file
- Add a basic html structure
- Add a heading using <h2> tag
- Insert the <video> tag with control attributes
- Specify the video file using <source> tag
- Save the file and open it the web browser
- Play the video using displayed controls.

My Video



Exp-3

write a program using Java script (arrays function)

Code:

```
let number = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray(arr) {
    Console.log("array Elements:");
    for (let i=0; i < arr.length; i++) {
        console.log(arr[i]);
    }
}

function findsum(arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++) {
        sum += arr[i];
    }
    return sum;
}

displayArray(numbers);
Console.log("sum=", findsum(numbers));
```

Output:

node "m:\array.js"

Array Elements:

2
45
8
23
56
19
34
1
90

Sum:278

```
JS array.js ●
M: > PYLAB > JS array.js > ...
1  let numbers = [2, 45, 8, 23, 56, 19, 34, 1, 90];
2  function displayArray(arr) {
3      console.log("Array Elements:");
4      for (let i = 0; i < arr.length; i++) {
5          console.log(arr[i]);
6      }
7  }
8  function findSum(arr) {
9      let sum = 0;
10     for (let i = 0; i < arr.length; i++) {
11         sum += arr[i];
12     }
13     return sum;
14 }
15 displayArray(numbers);
16 console.log("Sum:", findSum(numbers));

Problems  Output  Debug Console  Terminal  Ports
[Running] node "m:\PYLAB\array.js"
Array Elements:
2
45
8
23
56
19
34
1
90
Sum: 278
```

Ex-4

Develop dynamic webpage using Javascript

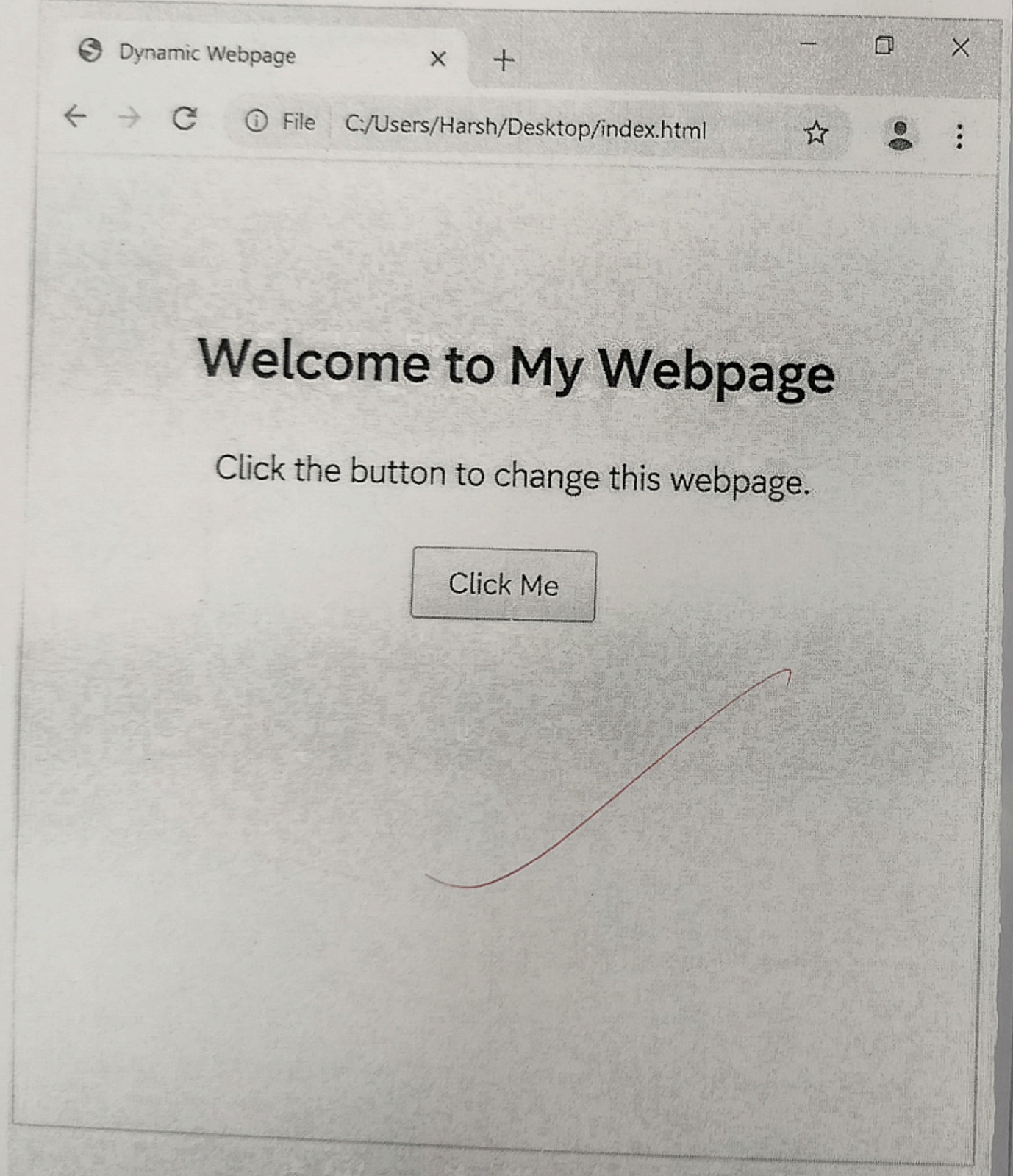
Code:

```
<!Doctype html>
<html lang="en">
<head>
<meta charset = "UTF-8">
<meta name = "viewport" content = "width = device-
    width, initial-scale = 1.0">
<title> Dynamic webpage </title>
<style>

body {
    font-family : Arial , Sans-serif ;
    text-align : center;
    margin :0 ;
    background-color : #f2f2f2 ;
}

header {
    background-color : #333 ;
    Color : White;
    Padding : 20 px;
}

Container {
    margin : 40px auto;
    Padding : 25 px;
    background : White;
    width : 80% ;
    max-width : 400 px;
```

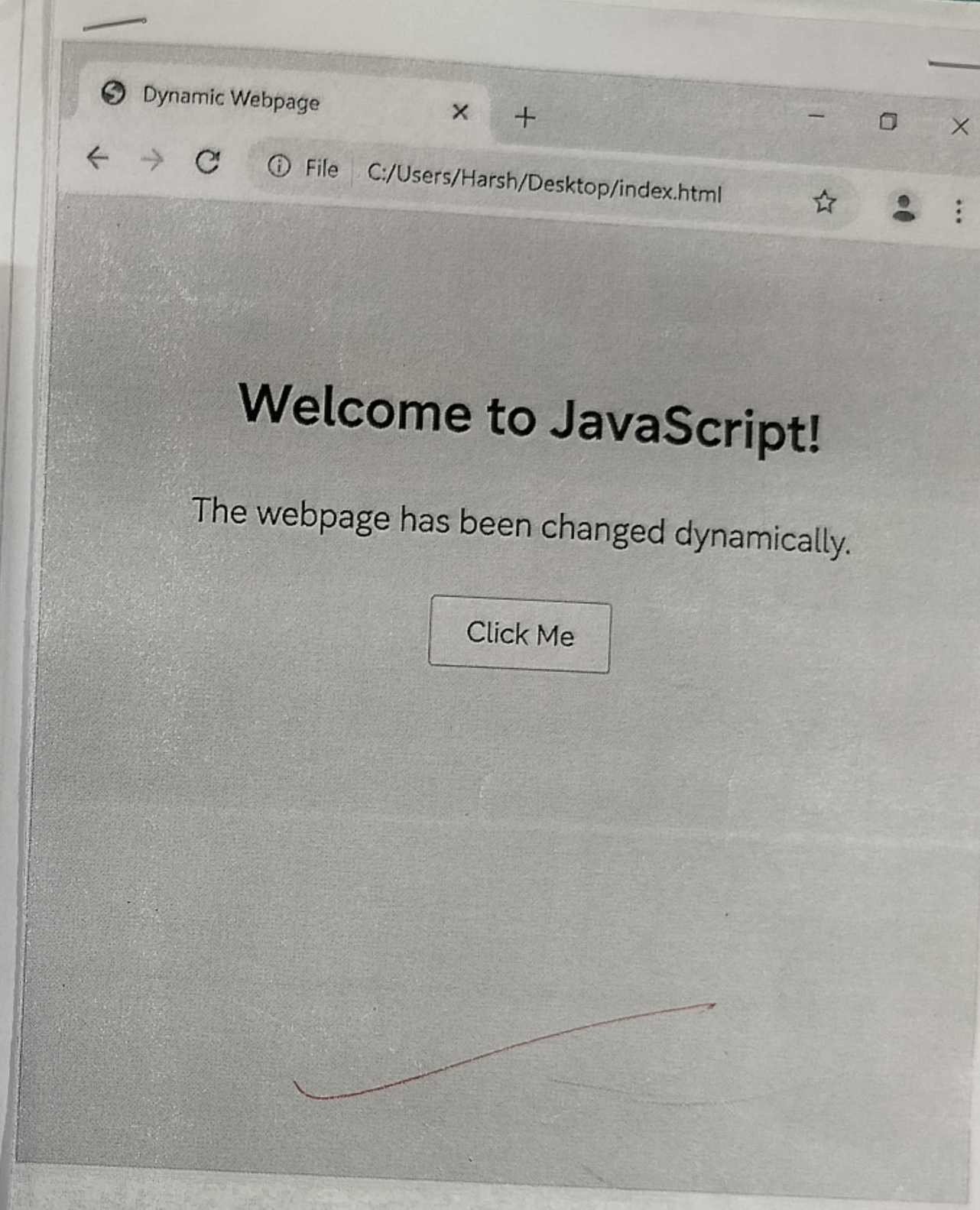
```
border-radius: 10px;
box-shadow: 0 0 10px gray;
}

input{
    Padding: 12 px;
    width: 85%;
    margin: 10px 0;
    border: 1 Px Solid # ccc;
    border-radius: 5 px;
}

button{
        Padding: 10px 20px;
        margin: 10 Px 5Px;
        border: none;
        border-radius: 5 Px;
        background-color: #333;
        Color: white;
        cursor: Pointer;
}

button: hover{
        background-color: #555
}

#message {
        margin-top: 20 px;
        font-size: 18 px;
        font-weight: bold;
}
```

```
</style>
</head>
<body>
 <header>
    <h1> Dynamic login page </h1>
    <P> Javascript Lab Demo </P>
</header>
<div class = "container">
  <h2> login </h2>
<input type = "email" id = "email" Placeholder = "Enter
    Your email" >
<input type = "password" id = "password" Placeholder
    = "Enter your password" >
<br>
<button onclick = "login()"> Submit </button>
<button onclick = "resetForm()" > Reset </button>
</div id = "message"> </div>
</div>
<script>
    function login() {
     let email = document.getElementById("email").value;
     let Password = document.getElementById("Password").value;
     let message = document.getElementById("message");
     if (email === "" || Password === "") {
     message.innerHTML = "Please enter both email and Password";
     message.style.Color = "red".
} else {
```

```
message.innerHTML = "Login Successful Welcome";
message.style.color = "given";
}

function resetform(){
```



Before clicking the button



After clicking the button

Experiment-5

5) Implement modules in Node Js

Source Code:

modules: Project/
- add.js
- Sub.js
- Mul.js
- div.js
- app.js

} → must be in same folder.

① add.js:

```
function add (a,b){
    return a+b;
}
module.exports = add;
```

② sub.js:

```
function sub (a,b) {
    return a-b;
}
module.exports = sub;
```

③ mul.js:

```
function mul (a,b){
    return a*b;
}
module.exports =mul;
```

④ div.js:

```
function div (a,b) {
    return a/b;
}
module.exports = div;
```

⑤ app.js:

```
const add = require ("./add");
const sub = require ("./sub");
const mul = require ("./mul");
const div = require ("./div");
let a=20;
let b=10;
console.log ("Addition:", add (a,b));
Console.log ("Subtraction:", sub(a,b));
Console.log ("Multiplication:", mul(a,b));
Consde.log ("Division:", div(a,b));
```

Bash: node app.js

Output:

Addition: 30
Substraction: 10
multiplication: 200
Division: 2

Experiment-6

1) Develop mini application using Express and Node.Js

Source Code & Process:

Install Node Js on your Computer after installation
Check it with
Bash
Node -v
npm -v

* Install Express:

npm install Express

Structure:

Express-mini.app/
- Express.js
- Javascript/
  - index.html

* The folder name Javascript must match the name used in .

Js:

app.use(Express.static('JavaScript'));

Express.Js

Const express = require('express');
Const app = express();
Const Port = 3000;

app.use(express.json());
app.use(express.static('JavaScript'));

app.post('/login', (req, res) => {
Const {email, password} = req.body;
if (email === 'admin@gmail.com' && Password === '12345'){
res.json({
success: true
message: login succesful'
});
}
else{
res.jsonc{
success: false
message: 'Invalid Email or Password!'
});
}
});

app.get('/about', (req, res) => {
res.send('About us page');
});

app.listen(Port, () => {
Console.log('server is running on Port 3000');
Console.log('visit http://Localhost:${Port}');

JavaScript/index.html:

<!Doctype html>
<html>
<head>
<title> Express mini application //title>
<Style>
body{

```
    font-family: Arial;
    text-align: center;
    margin-top: 80px;
    background-color: #f2f2f2
}

.container {
    background: white;
    width: 350px;
    margin: auto;
    Padding: 30px;
    border-radius: 10px;
    box shadow: 0 0 10px gray;
}

input{
    width: 90%;
    Padding: 10px;
    margin: 10px;
}

button{
    Padding: 10px 20px;
    margin: 10px;
    Cursor: Pointer;
}

#result{
    margin-top: 20px;
    font-weight: bold;
}
</style>
</head>

<body>
```

```
<div class = "container">
  <h2> login </h2>

<input type = 'email" id="email" placeholder = "Enter email" >
<input type = "Password" id= "Password" Placeholder = "Enter Password">
<br>

<button onClick = "login()" > login </button>
<button onClick = "resetform()" > Reset </button>
<p id = "result" > </p>
<a href = "/about" > about us </a> </div>

<script>
  function login() {
    Const email = document.getElementById("Email"). value
    Const Password = document.getElementById("Password"). value
    fetch ('/login', {
        method : 'POST')
        headers : {
            'Content-Type' : 'application/json'
    },
    body : Json.Stringify({
        email: email
        Password: password
    })
})
  .then (response => response.json())
  .then(data => {
    document.getElementById("result").innerText =data.message;
});
}

function resetform(){
```

```
document.getElementById("email").value = "";
document.getElementById("password").value = "";
document.getElementById("Result").value = "";
}
</script>
</body>
</html>
```

Commands to execute:

Cd express-mini-app

Install Express

npm install express

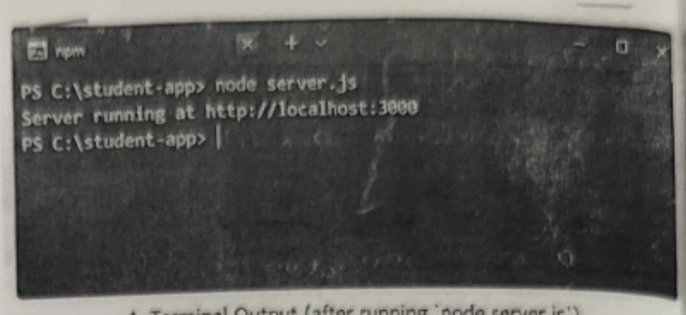
Start the server

node express.js

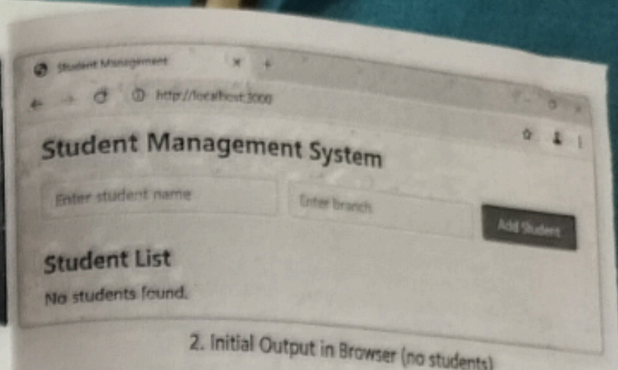
You should see:

Server is running on Port 3000

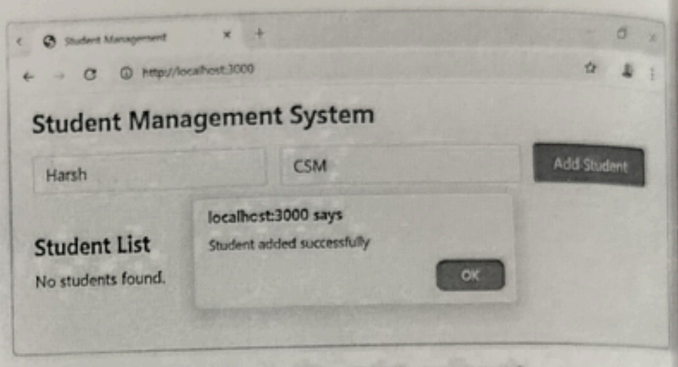
visit http://localhost:3000



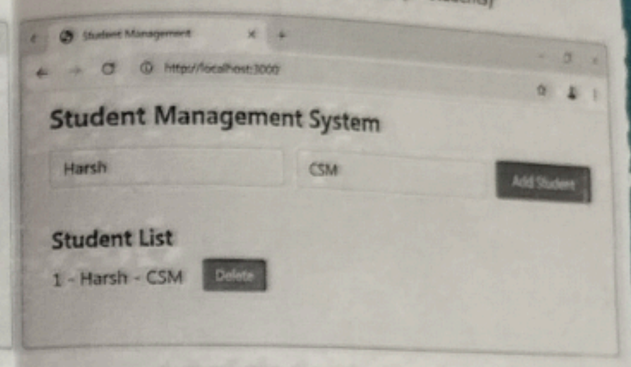
1. Terminal Output (after running `node server.js`)



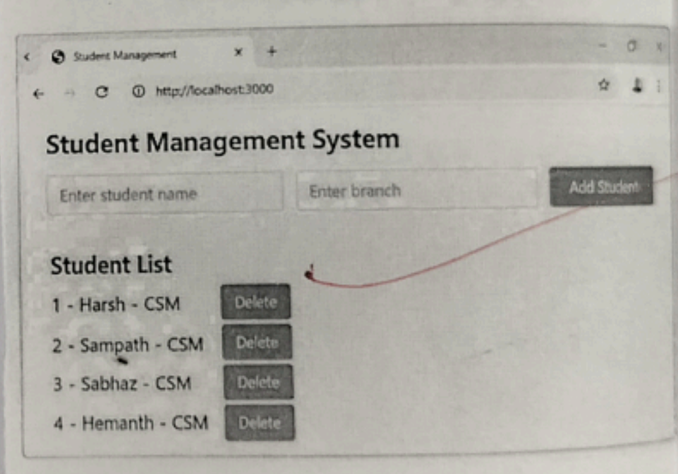
2. Initial Output in Browser (no students)



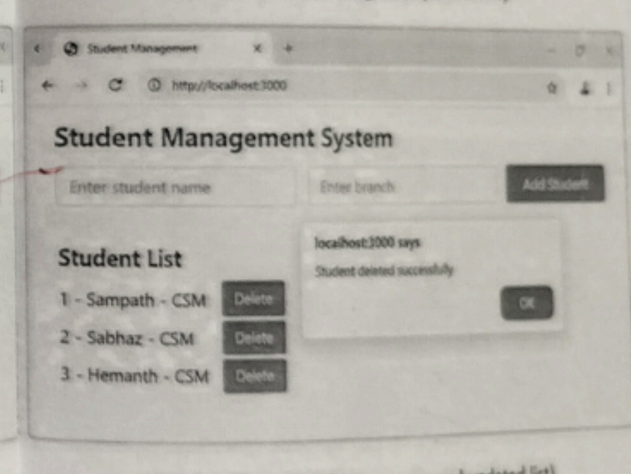
3. After adding Harsh (alert message)



4. Student list after adding Harsh (main name)



5. After adding all friends (Harsh, Sampath, Sabhaz, Hemanth)



6. After deleting Harsh (alert message and updated list)

Experiment-7

Create a functional Component and class Component using react.

Aim: To develop a react application using function Components and class Components to display student details, manage State. handle button clicks and dynamically update data on a webpage.

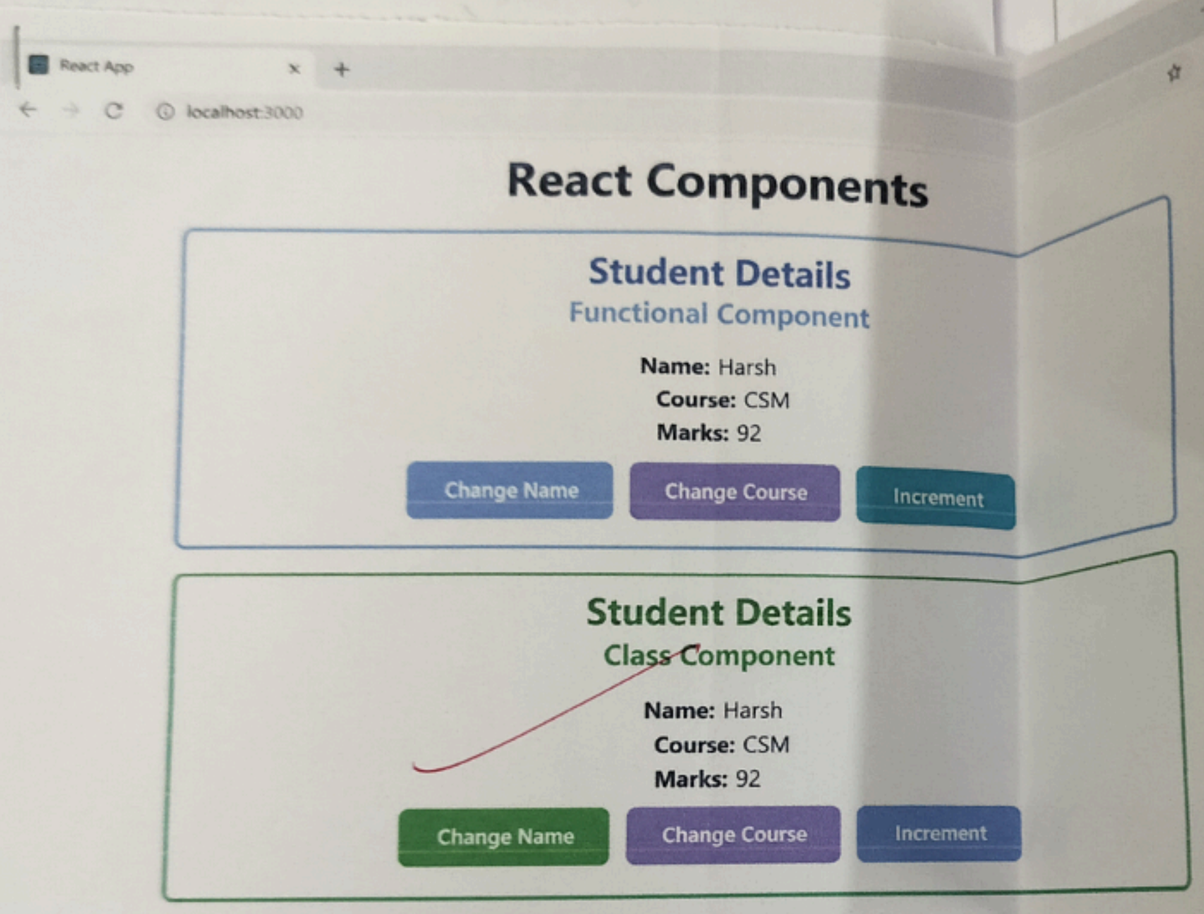
- Functional Component

A functional Component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage State and update the webpage dynamically

Program

```
import React {useState} from 'react';

function StudentFunctional ( ) {
    Const [name, setName] = use State ("Harsh");
    Const [course, SetCourse] = useState ("CSM");
    Const [count, setCount] = useState (0);

    Const  changeName = ( ) => {
                SetName ("Raj");
    };

    Const ChangeCourse = ( ) => {
                setCourse ("CSD")
    };
    const  incrementCount = ( ) => {
                Set (count+1) ;
    };
    return (
        <div style = {{
            border : "2px Solid blue";
            Padding : "20px";
            margin : "20px";
            borderRadius : "10px";
            textAlign : "Center"
        }} >
        <h1 > Student Details </h1>
        <h2 > Functional Component </h2>
        <P><b > Name : </b> {course} </P>
        <P > <b > Button Clicks : </b> {count} </P>
        <button onClick = { changeName } >
                ChangeName </button>
        <button onclick = { changeCourse } changeCourse
                </buttons >
        <button  onclick = {incrementCount} > Increment
                </button >
        </div >
        );
```

o/p: when the user clicks the buttons, the name Course and Count Change dynamically without reloading the Webpage.



Class Component:

A Class mponent is an ESE JavaScript
Class xtends React.Component, It uses the
method to return JSX and manages
this state and this.setState().

```
t {Component} from 'react';
- (class extends Component {
tor (Props){
pes (Props);
is.state={
  name: "Harsh",
  Course: "Computer Science",
  Count: 0
};

e Name=() => {
  this.setState({
    name: "Raj"
  });
};
Change Course = () => {
  this.setState({
    course: "CSM"
  });
};
increment(ount = () => {
  this.sdState((PrevState) => ({
    Count. PrevState. Count + 1
```

```
        });
    };
    render() {
      return (
      <div style={{
            border: "2px solid green",
            Padding: "20px",
            margin: "20 px",
            borderRadius: "10px",
            textAlign: "center",
          }}>
      <h1> Student Details </h1>
      <h2> Class Component </h2>
      <p> <b> Name: </b> {this.State.name} </p>
      <p> <b> Course: </b> {this.State.course} </p>
      <p> <b> Button Clicks: <b> {this.State.count} </p>
      <button onClick={this.changeName}>
            ChangeName </button>
      <button onClick={this.changeCourse}>
            ChangeCourse </button>
      <button onClick={this.incrementCount}>
            increment </button>
      </div>
```

Experiment-8

8) Implement http methods using Json Server

Aim: To implement HTTP method such as get, Post, put, Patch and delete using Json Server to Perform CRUD operation on Student records.

Theory: JSON Server is a tool used to create a fake Rest API using a Json file. It allows us to Perform HTTP opeation without developing a backend Server from search

HTTP methods are used to communicate b/w a client and a Server.

- GET – retrieves data from the Server
- POST – Adds new data to the Server
- PUT – Replaces an existing record
- PATCH – Updates selected fields of a record
- Delete – removes a record from the server

a) Create a JSON server:

Step1: Initialize the Project

Open the vs code terminal and execute the following commands:

```
mkdir Json server
Cd Json Server
npm init -Y
npm install json server
```

Step2: Create db-json

Create a file named db-json and add the following student data

```
{
  "students":
    {
      "id": "1",
      "name": "Harsh",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "Shabaz",
      "course": "CSM",
      "marks": "90",
    }
    {
      "id": "3",
      "name": "Sampath",
      "course": "CSM",
      "Marks": "99"
    }
```

Step 3: Start the JSON server for the current JSON server version, execute

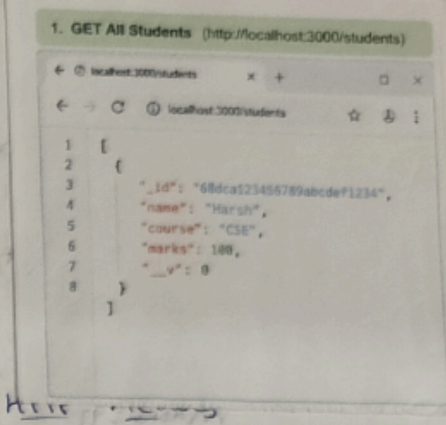
```
npx json-server db.json
```

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students.

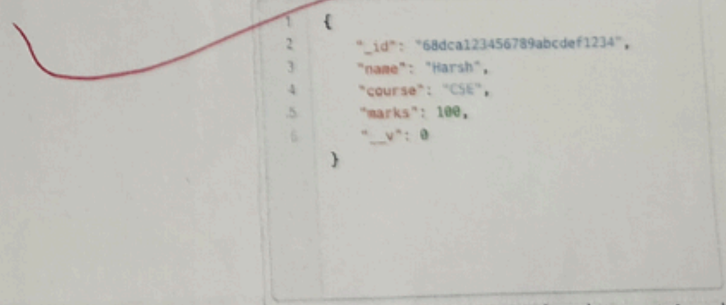


2. Implementation of HTTP requests

a) GET method: The GET method is used to retrieve Student records from Json server code:

Code:

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  })
```

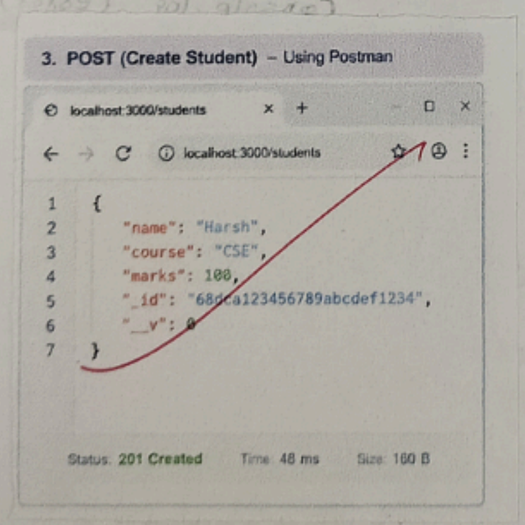


b) Post Method: The Post method is used to insert a new student record into the server.

Code: fetch ('http://localhost:3000/students', {

```
    method: 'POST'
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: Rahul,
        course: CSPM,
        marks: 92
    })
})

.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error as {
    console.log(error);
});
```

O/P:

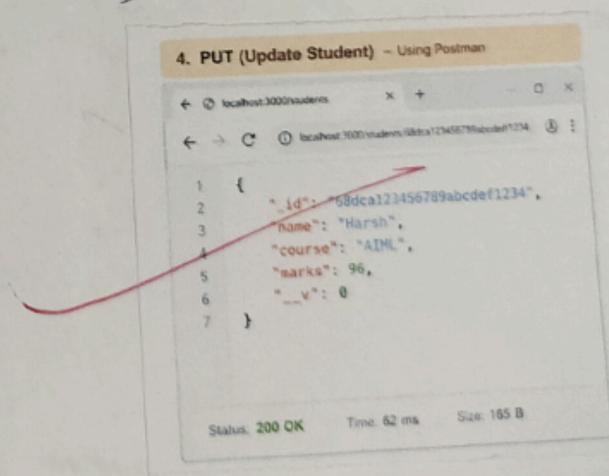


c) PUT method: The PUT method is used to replace an existing student record with updated data

Code: fetch ('http://localhost:3000/students/1', {

```
    method: 'PUT',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Jeyaswasi',
        Course: 'AIML',
        marks: 95
    })
})

.then (response => response.json())
.then (data => {
    console.log(data);
})
.catch (error => {
    console.log(error);
});
```

O/P:



d) Patch method: The Patch method is used to update only selected fields of an existing student

record.

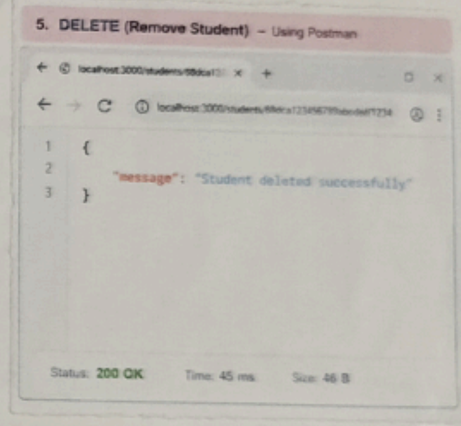
Code: fetch ('http://localhost:3000/students/2', {
  method: 'patch',
  headers: {
    'content-Type': 'application/json'
  },
  body: JSON.stringify ({
    marks: 96
  })
})
.then (response => response.json())
.then (data => {
  console.log (data);
})
.catch (error => {
  console.log (error);
});

O/P:

E) Delete method: The delete method is used remove an existing student record from the ser

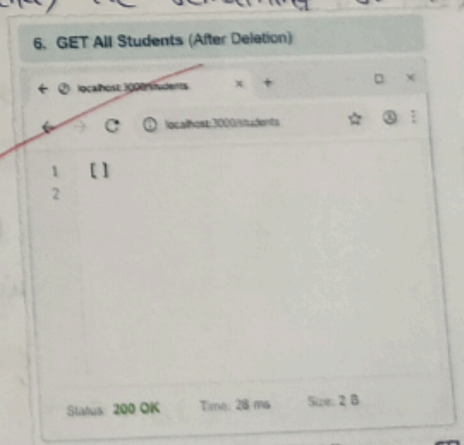
Code: fetch ('http://localhost:3000/students/3', {
  method: 'Delete'
})
.then response => {
  if (response.ok){
    Console.log ("student deleted success");
  }}
.catch (error => {
  console: log (error)
});

O/P:



After executing the POST, PUT, PATCH & DELETE operations in sequence, the remaining student will be:

O/P:



Thus HTTP methods like delete were successfully implemented using JSON server. The CRUD operations were performed on student records. The data was retrieved, inserted, updated & deleted using HTTP requests.

Experiment - 9

9) Develop CRUD operations using MongoDB

Aim: To develop a web application using Node.js and MongoDB to perform CRUD operations (Create, Update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores flexible, JSON like documents CRUD operations are Manage data in a database.

Mongoose is an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define Schemas, interact with Collection and database operations.

Setup:

1) Installation & Project

```
MKdir MongoDB-CRUD
cd MongoDB-CRUB
npm init -y
npm install express mongoose
```

2) Platform: server.js

```
Const express = require("express");
Const mongoose = require('mongoose');
Const app = express();
```

Code: fetch ('http://localhost:2000/students/3', {

```
    method: 'Delete'
})
.then (response => {
    if (response.ok) {
        Console.log("student deleted success");
    }
})
.catch (error => {
        Console.log (error)
});

Const PORT = 3000;
app use (express.json());
mongoose Connect ('mongodb://127.0.0.1:
                  2017/college)
.then (() => console.log ("MongoDB Connected"))
.catch (err => Console.log(err));

//Create student
Const  Student Schema = new
mongoose.schema ({
    name: {
        type: string,
        requirement: true
    },
    Course: {
        type: string
```

required: true

},

marks: {

type: Number

required: true

}

});

// create Student model.

Const Student:

mongoose.model('student', studentSchema)

//create: Add

app.post ('/students', async (req.res) => {

try {

Const student = new Student (req.body);

Const savedStudent = await Student save();

res.Status (201).json (savedStudent);

}catch (error){

res.status (400).json({message:

error: message});

}

});

//READ: Get all students

app.get ('/students', async(req.res) => {

try {

const Students : await

Student.Find ();

res.status(500).json ({message: error.message});

}

});

//READ: Get Student by ID

app.get ('/student = 'id', async (req.res) => {

try{

Const Student: await

Student findById (req.params.id);

if (! student){

return.resStudent (404).json ({

message: "student not found",

});

}

res.status (200).json (student)

} catch (error){

res.status (400).json ({message:

error.message});

}

});

//update

app.Put ('/students/:id'.async (req.res => {

try {

Const Student: await

Student.findByIdandUpdate (

req.params.id,
req.body,
{ new: true, runValidators: true }
);

if (!student) {
return res.status(404).json({
message: "Student not found"
});
}

res.status(200).json(student);
} catch (error) {
res.status(400).json({ message:
error.message });
}
});

//Delete

app.delete('/students/:id', async (req, res) => {
try {
const student = await Student.
findByIdAndDelete(
req.params.id
);
if (!student) {
return res.status(404).json({
message: "student not found"
});
});

res.status(200).json({
message: "student deleted successfully"});
}
catch (error){
res.status(400).json({ message: error.message })
}
});

O/P: